

AWS vs GCP

Data Engineering Service Mapping

Quick Interview Reference Guide

Prepared for: Principal Data Engineer Interview

Focus: AWS + Databricks Ecosystem

Table of Contents

1. Storage (S3 vs GCS)
2. Metadata/Catalog (Glue Catalog vs Dataplex)
3. Data Warehouse (Redshift vs BigQuery)
4. Batch ETL/Spark (Glue/EMR vs Dataproc)
5. Streaming ETL
6. Orchestration (MWAA vs Composer)
7. Serverless Compute (Lambda vs Cloud Functions)
8. Container Orchestration
9. Message Queues (SNS/SQS vs Pub/Sub)
10. Streaming Platform (Kinesis vs Kafka)
11. Security & Secrets
12. Monitoring & Logging
13. Lakehouse Platform (Databricks)
14. BI Tools
15. Query Engines (Athena)
16. AWS-Only Services
17. Interview Translation Cheat Sheet
18. Databricks Deep Dive
19. Final Interview Strategy

1. STORAGE

GCP	AWS	Key Differences
Google Cloud Storage (GCS)	Amazon S3	95% identical concepts

Must-Know Points for S3:

- Object storage with buckets and prefixes (no real folders)
- 7 storage classes for cost optimization (Standard, IA, Glacier, etc.)
- Strong consistency for all operations (since Dec 2020)
- Plan prefix distribution for high throughput (5,500 req/sec per prefix)
- Lifecycle policies for automatic tier transitions
- Encryption: SSE-S3 (AWS-managed), SSE-KMS (customer-controlled), SSE-C
- Versioning for data protection and audit trails
- S3 Select for efficient querying without downloading entire files

Interview Answer Template:

"S3 is nearly identical to GCS. Both use object storage with buckets and prefixes. I've architected petabyte-scale data lakes on GCS using partition strategies and lifecycle policies - same concepts apply to S3. Main difference is S3 has more storage classes (7 vs 4) for granular cost optimization."

2. METADATA / CATALOG

GCP	AWS	Key Differences
Dataplex Catalog BigQuery Metadata	AWS Glue Data Catalog	GCP more integrated, AWS more universal

AWS Glue Data Catalog:

- Central metastore shared across Athena, Redshift Spectrum, EMR, Glue Jobs
- Stores databases, tables, schemas, partitions
- **Crawlers** auto-discover schema from S3, RDS, Redshift
- Partition management (add/drop partitions automatically)
- Think of it as: Hive Metastore as a managed service
- Integration: Define table once, query from multiple services

Interview Answer:

"Glue Catalog is a shared metastore similar to Dataplex. When you define a table pointing to S3 data, both Athena and Redshift Spectrum can query it using the same metadata. Crawlers auto-discover schema changes, which is useful for evolving data sources."

3. DATA WAREHOUSE

Feature	BigQuery (GCP)	Redshift (AWS)
Pricing	Pay per query (TB scanned)	Pay for nodes 24/7
Management	Serverless, zero tuning	Manual optimization required
Scaling	Instant, automatic	Resize cluster (downtime)
Storage	Decoupled from compute	Tied to compute (RA3 can separate)
Tuning	Auto-optimized	DISTKEY, SORTKEY, VACUUM
Best For	Unpredictable workloads	Predictable, cost-sensitive workloads

Redshift Key Concepts:

- **Distribution Keys (DISTKEY):** Colocate related data on same nodes for fast joins
- **Sort Keys (SORTKEY):** Physical ordering for fast range queries (like BigQuery clustering)
- **VACUUM:** Reclaim space after DELETES, re-sort data (BigQuery doesn't need this)
- **ANALYZE:** Update statistics for query optimizer
- **COPY command:** Fastest way to load from S3 (parallel, compressed)
- **Redshift Spectrum:** Query S3 directly without loading (like BigQuery external tables)
- **Node types:** DC2 (fast), RA3 (storage-compute separation)

Interview Answer:

"I'm coming from BigQuery which is serverless. I understand Redshift requires manual optimization - distribution keys ensure data colocation for fast joins, sort keys enable efficient range queries, and VACUUM maintains performance. The trade-off is cost predictability (fixed cluster cost) versus BigQuery's elasticity."

4. BATCH ETL / SPARK

GCP	AWS	Use Case
Dataproc (Managed Hadoop/Spark)	EMR (Elastic MapReduce)	Full Spark control, complex tuning
Dataproc Serverless	AWS Glue Jobs	Serverless Spark ETL, simple jobs

AWS Glue Jobs - Key Points:

- Serverless Spark - no cluster management
- **DPU (Data Processing Unit):** 4 vCPU + 16 GB memory + 64 GB disk
- Billing: Per-second DPU usage (1-minute minimum)
- **Job Bookmarks:** Track processed data for incremental loads
- **DynamicFrame:** Schema-flexible abstraction over Spark DataFrame
- Startup time: ~1 minute (slower than Databricks ~30 sec)
- When to use: Simple ETL, tight AWS integration
- When NOT to use: Complex Spark tuning, cost-sensitive at scale

Glue vs EMR vs Databricks:

Glue: Fully serverless, simple ETL, higher cost per job. **EMR:** Full control, complex tuning, cheaper at scale with Spot instances. **Databricks:** Modern platform, fastest startup, best optimization (Photon, AQE).

5. ORCHESTRATION

GCP	AWS	Notes
Cloud Composer (Managed Airflow)	MWAA (Managed Workflows for Apache Airflow)	99% compatible - same DAG code

Key Points:

- Your Cloud Composer DAGs transfer directly to MWAA
- Same Airflow operators, connections, variables
- Alternative: Databricks Workflows (for Databricks-specific pipelines)
- AWS Step Functions: JSON-based state machine (less flexible than Airflow)

Interview Answer:

"I've used Cloud Composer extensively for orchestrating complex ETL workflows. AWS MWAA is the same managed Airflow service - my DAG code would transfer with minimal changes. For Databricks-specific workflows, I'd evaluate Databricks Workflows versus Airflow based on complexity and team familiarity."

6. SERVERLESS COMPUTE

GCP	AWS	Similarities
Cloud Functions	Lambda	99% identical - event-driven, 15-min timeout

- Event-driven compute (S3, SNS, EventBridge triggers)
- 15-minute maximum timeout
- Cold start considerations (provisioned concurrency for critical paths)
- Lambda Layers for shared dependencies
- Use cases: Lightweight transformations, pipeline triggers, data validation

7. MESSAGE QUEUES / PUB-SUB

GCP	AWS	Notes
Pub/Sub	SNS (pub-sub) SQS (queue)	AWS splits into two services

- **SNS:** Publish to multiple subscribers (broadcast)
- **SQS:** Queue for one consumer (reliable delivery)
- **Pattern:** SNS → multiple SQS queues (fan-out)
- Use SNS for event notifications, SQS for decoupled processing

8. DATABRICKS PLATFORM (CRITICAL)

What is Databricks?

Unified analytics platform built on Apache Spark. Provides Delta Lake (ACID on data lakes), Unity Catalog (governance), and Delta Live Tables (declarative pipelines). Runs on AWS infrastructure.

Core Components:

Component	Description	Key Features
Delta Lake	ACID transactions on S3 using transaction log	- Time travel (query historical versions) - Schema evolution - MERGE operations (upserts) - OPTIMIZE & Z-ORDER for performance
Delta Live Tables (DLT)	Declarative pipeline framework	- Define WHAT you want, not HOW - Built-in data quality (expectations) - Automatic dependency resolution - APPLY CHANGES for CDC
Unity Catalog	Centralized governance	- Fine-grained access control - Automatic lineage tracking - Audit logging - Cross-workspace metadata
Photon Engine	Vectorized query execution	2-10x faster than standard Spark - Automatic optimization - No code changes needed

Delta Lake vs Traditional Data Lake:

Feature	Traditional (Parquet on S3)	Delta Lake
ACID	■ No	✓ Yes
Time Travel	■ Manual versioning	✓ Built-in (query old versions)
Schema Evolution	■ Break pipelines	✓ Safe column additions
Upserts/Deletes	■ Rewrite entire partition	✓ Efficient MERGE
Consistency	■ Eventual	✓ Strong (transaction log)

Delta Live Tables (DLT) - THE Core Skill

Declarative Pipelines: Define tables using SQL or Python, DLT handles execution

Data Quality (Expectations):

- WARN: Log violations, continue processing
- DROP: Quarantine bad rows
- FAIL: Stop pipeline if constraint violated

Incremental Processing:

- Streaming tables: Real-time processing
- APPLY CHANGES: Handle CDC (Change Data Capture) automatically

Auto-Optimization: DLT handles performance tuning, scaling

vs Traditional Spark:

- Traditional: Write explicit DAG, handle dependencies, implement data quality
- DLT: Declare what you want, DLT figures out how

Interview Translation:

Your Cloud Composer DAGs with manual dependency management → DLT with automatic dependency resolution

Your custom data validation logic → DLT Expectations

Your retry/recovery logic → DLT auto-recovery

Databricks Cost Optimization

- **Job clusters vs Interactive clusters:** Use ephemeral job clusters (terminate after job)
- **Auto-scaling:** Enable for variable workloads
- **Spot instances:** Use for fault-tolerant batch jobs (60-90% savings)
- **Photon:** Costs 2x DBUs but 2-10x faster (net savings)
- **Monitor DBU consumption:** Track by job, user, cluster type
- **Right-size clusters:** Don't over-provision cores/memory

Interview Question: 'Databricks bill increased 3x, how do you diagnose?'

Answer: Check DBU consumption metrics by cluster type - are interactive clusters running 24/7? Are jobs using job clusters or interactive? Check for inefficient queries causing long runtimes. Verify auto-scaling is enabled. Look for zombie clusters not terminating. Compare Photon vs standard compute - Photon costs more but may complete faster.

9. INTERVIEW TRANSLATION CHEAT SHEET

They Ask	You Respond
Have you used AWS Glue?	I've used Dataproc Serverless which is similar - serverless Spark for ETL. Glue adds job bookm
Experience with S3?	I've architected petabyte-scale data lakes on GCS, which is nearly identical. Same concepts: pa
Redshift experience?	I'm coming from BigQuery which is serverless. I understand Redshift requires manual optimizati
How would you orchestrate?	I've used Cloud Composer (managed Airflow) extensively. AWS MWAA is the same - my DAG o
Databricks experience?	While I haven't used Databricks in production, I have deep Spark expertise and understand the

10. FINAL INTERVIEW STRATEGY

Opening Statement (When They Ask About AWS):

"My background is primarily GCP, but core data engineering principles are cloud-agnostic. I've architected 40 PB migrations, built configuration-driven frameworks, and optimized Spark at scale - these skills transfer directly. I've done deep research on AWS equivalents: S3 maps to GCS, Glue to Dataproc, Airflow works on both. The learning curve is short given my foundation."

When You Don't Know Something:

"I haven't used [AWS Service X] hands-on, but I understand it's conceptually similar to [GCP equivalent]. In my previous role, I ramped up on [new technology] in [timeframe] and became the team expert. I'm confident I can do the same here."

Emphasize Your Strengths:

- ✓ **40 PB migration** (most candidates don't have this scale)
- ✓ **Configuration-driven frameworks** (platform thinking, not just pipelines)
- ✓ **Production support for 60+ teams** (reliability, stakeholder management)
- ✓ **Security/governance in banking** (compliance, audit, encryption)
- ✓ **Multi-source ingestion** (Teradata, Oracle, Hadoop, Kafka - migration complexity)

High-Impact Stories to Prepare:

1. **40 PB Migration:** What went wrong? How did you fix it? How did you validate correctness?
2. **Performance Optimization:** Slowest job you optimized? What was the bottleneck? How did you diagnose?
3. **Failure Recovery:** Biggest production incident? Root cause? How did you prevent recurrence?
4. **Stakeholder Management:** How did you get 60+ teams to adopt your framework?
5. **Architecture Decision:** Trade-off you made that you'd do differently now?

Questions to Ask Them:

- What's your current Databricks maturity? (pilot vs production-scale)
- Are you migrating from on-prem or already on AWS?
- Biggest pain point in current data platform?
- How do you balance time-to-market vs technical excellence?
- What does success look like in first 90 days?

11. QUICK REFERENCE TABLE

Category	GCP	AWS	Key Difference
Storage	GCS	S3	S3: 7 storage classes vs GCS 4
Warehouse	BigQuery	Redshift	BigQuery serverless, Redshift manual tuning
Batch Spark	Dataproc	EMR/Glue	Glue serverless, EMR full control
Catalog	Dataplex	Glue Catalog	Both: shared metastore
Orchestration	Composer	MWAA	Same: Apache Airflow
Serverless	Functions	Lambda	Identical: 15-min limit
Pub/Sub	Pub/Sub	SNS+SQS	AWS splits into 2 services
Streaming	Dataflow	Kinesis	Kinesis: Kafka-like
Lakehouse	BigLake	Databricks	Databricks: THE platform
Query	BigQuery	Athena	Athena: query S3 only

Remember:

You're 60% ready based on transferable skills. Focus on Databricks concepts (DLT, Delta Lake, Unity Catalog) and your migration stories. Your 40 PB migration is your superpower - most candidates don't have that scale.

Good luck! ■